

Pocket – Backend-Anforderungen für Spring Boot

Dokumenttyp: Schnittstellen- und Implementierungsanforderungen

Frontend-Basis: `index(3).html` (Pocket | Finanzverwaltung)

Ziel: Ablösung von Demo-/localStorage-Daten durch ein persistentes REST-Backend auf Spring Boot.

1. Kurzüberblick und Analyse des Frontends

Die HTML-Anwendung ist eine Single-Page-Finanzverwaltung mit diesen Bereichen:

- **Login/Logout:** E-Mail und Passwort; derzeit hart codierter Demo-Login (`demo@pocket.app / demo123`).
- **Dashboard:** Gesamtsaldo, Einnahmen, Ausgaben, monatliches Netto wiederkehrender Buchungen, Einnahmen-/Ausgaben-Verlauf und letzte Aktivitäten; Button zum Erstellen einer Transaktion.
- **Transaktionen:** Anlegen, Bearbeiten, Löschen; Suche nach Beschreibung/Kategorie; Filter Alle, Einnahmen, Ausgaben, wiederkehrend, einmalig.
- **Statistiken:** Kennzahlen wiederkehrender Einnahmen/Ausgaben, Netto, Auflistungen wiederkehrender Posten, Fixkosten-vs.-variable-Ausgaben sowie Kategorien-Auswertungen und Charts.
- **Kategorien:** Kategorien werden derzeit im Frontend als Liste verwaltet und für die Transaktionsauswahl verwendet.
- **Einstellungen:** Vier Themes (`light, dark, ocean, midnight`). Theme ist eine reine UI-Einstellung und kann zunächst im Browser bleiben.

Der aktuelle Datenzustand liegt überwiegend in `localStorage`

(`pocket_transactions`, `pocket_auth_user`, `pocket_theme`). Bei leerem Speicher werden Demo-Transaktionen geladen. Die Backend-Anbindung muss Demo-Daten im Produktivbetrieb vollständig ersetzen.

Wichtiges fachliches Modell

Im aktuellen Frontend sind wiederkehrende Buchungen normale Transaktionen mit `recurring=true` und `recurringInterval="monthly"`. Das ist für eine erste Migration kompatibel, aber fachlich sollten **wiederkehrende Regeln (Vorlagen)** von **tatsächlich gebuchten Transaktionen** getrennt werden. Sonst würden z. B. monatliche Miete und Gehalt nur als ein einzelner Datensatz existieren oder beim erneuten Generieren doppelt gebucht.

Empfohlene Umsetzung:

- `RecurringRule`: wiederkehrende Einnahme/Ausgabe mit Betrag, Kategorie, Intervall, Start-/Enddatum und aktiv/inaktiv.
- `Transaction`: konkrete Buchung mit Datum; optionaler Verweis auf die erzeugende Regel.
- In der UI als „Fixe/wiederkehrende Einnahmen/Ausgaben“ anzeigen. Beide Typen müssen unterstützt werden.
- Für die erste Backend-Version kann ein `recurring`-Flag an einer Buchung zusätzlich als Legacy-Feld akzeptiert werden, sollte aber nicht die langfristige Datenstruktur sein.

2. Technische Rahmenbedingungen

- Spring Boot REST API, JSON über HTTPS.
- API-Präfix: `/api/v1`.
- Java 21+ (oder Projektstandard), Spring Web, Spring Security, Spring Data JPA, Bean Validation.
- Persistenz: MySQL; Beträge als `DECIMAL(19,2)`/Java `BigDecimal`, niemals `float` oder `double`.
- Datumswerte als ISO-8601 `YYYY-MM-DD`; Zeitstempel als ISO-8601 mit Zeitzone/UTC.
- IDs als UUID (String im JSON).

- CORS nur für die tatsächliche Frontend-Origin freigeben; keine Wildcard mit Credentials.
- Alle Datenzugriffe müssen auf den authentifizierten Benutzer eingeschränkt sein.
- Einheitliches Fehlerformat (siehe Abschnitt 8).
- API-Dokumentation via OpenAPI/Swagger empfohlen.

3. Datenmodell (Vorschlag)

User

Feld	Typ	Regeln
id	UUID	Primärschlüssel
name	String	Pflicht, 1–100 Zeichen
email	String	Pflicht, eindeutig, normalisiert/ lowercase
passwordHash	String	BCrypt/Argon2; nie ausgeben
createdAt, updatedAt	Instant	serverseitig

Category

Feld	Typ	Regeln
id	UUID	Primärschlüssel
userId	UUID	Eigentümer; alternativ systemweite Standardkategorien plus benutzerspezifische Kategorien
name	String	Pflicht, 1–60 Zeichen; innerhalb eines Benutzers eindeutig
type	enum	INCOME, EXPENSE, BOTH (empfohlen)

color	String?	optionales UI-Merkmal, z. B. Hex-Farbe
archived	boolean	archivierte Kategorien bleiben für alte Buchungen erhalten

Die bisherigen Namen enthalten u. a. Gehalt, Nebeneinkommen, Wohnen, Bills, Essen & Trinken, Mobilität, Freizeit, Hobby, Shopping und Sonstiges. Standardkategorien sollten initialisiert werden können, aber nicht bei jedem Login erneut dupliziert werden.

Transaction

Feld	Typ	Regeln
id	UUID	Primärschlüssel
userId	UUID	Eigentümer aus Auth-Kontext, nie aus Request übernehmen
description	String	Pflicht, 1–160 Zeichen
amount	BigDecimal	Pflicht, > 0, max. 2 Nachkommastellen
type	enum	INCOME oder EXPENSE
categoryId	UUID	Pflicht; muss dem Benutzer gehören und zum Typ passen (BOTH erlaubt)
transactionDate	LocalDate	Pflicht
recurringRuleId	UUID?	optionaler Bezug zur wiederkehrenden Regel
createdAt, updatedAt	Instant	serverseitig

Die UI erwartet positive Beträge und unterscheidet Einnahme/Ausgabe separat. Daher Betrag positiv speichern; Vorzeichen erst in Saldo-Berechnungen berücksichtigen.

RecurringRule (Fixe/wiederkehrende Buchung)

Feld	Typ	Regeln
id	UUID	Primärschlüssel
userId	UUID	Eigentümer
description	String	Pflicht
amount	BigDecimal	> 0, max. 2 Nachkommastellen
type	enum	INCOME oder EXPENSE – beide ausdrücklich unterstützen
categoryId	UUID	Kategorie des Benutzers
interval	enum	mindestens WEEKLY, MONTHLY, YEARLY; Erweiterung möglich
startDate	LocalDate	Pflicht
endDate	LocalDate?	optional, inklusiv oder klar dokumentiert exklusiv festlegen
nextOccurrenceDate	LocalDate?	falls automatische Buchung implementiert wird
active	boolean	deaktivierte Regel erzeugt keine zukünftigen Buchungen
createdAt, updatedAt	Instant	serverseitig

Abgrenzung: Eine Regel ist nicht automatisch eine bereits erfolgte Zahlung. Das Dashboard kann die Summe aktiver Regeln als „wiederkehrendes Netto pro Monat“ darstellen. Für tatsächliche Monatsauswertungen zählen dagegen nur konkrete Transaktionen.

4. REST-Schnittstellen

Alle geschützten Endpunkte benötigen gültige Authentifizierung. JSON-Requests und -Responses verwenden `Content-Type`:

application/json.

4.1 Authentifizierung

Methode	Pfad	Zweck
POST	/api/v1/auth/register	Benutzerkonto erstellen (falls Registrierung gewünscht)
POST	/api/v1/auth/login	Anmelden und Token/Session erhalten
POST	/api/v1/auth/logout	Abmelden; bei JWT optional clientseitig, bei Refresh-Token serverseitig widerrufen
GET	/api/v1/auth/me	Aktuellen Benutzer für Sidebar/Login-Status liefern

Login Request:

```
{ "email": "user@example.com", "password": "secret" }
```

Login Response (JWT-Variante):

```
{
  "accessToken": "<jwt>",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": { "id": "uuid", "name": "Demo User", "email": "
}
```

Sicherheit: Demo-Zugangsdaten entfernen; keine Passwörter im Klartext speichern/loggen. Frontend darf nicht selbst einen Benutzer als authentifiziert in `localStorage` festlegen. Bevorzugt `HttpOnly/Secure/SameSite-Cookie` oder kurzlebiges Access-Token mit sicherem Refresh-Verfahren. Bei JWT muss die konkrete Token-Speicherung im Frontend bewusst festgelegt werden.

4.2 Transaktionen

Methode	Pfad	Zweck
GET	/api/v1/ transactions	Liste mit Filter, Suche, Sortierung und Pagination
GET	/api/v1/ transactions/{id}	Einzelne Buchung abrufen
POST	/api/v1/ transactions	Buchung erstellen
PUT	/api/v1/ transactions/{id}	Buchung vollständig aktualisieren
PATCH	/api/v1/ transactions/{id}	Optional: Teiländerung
DELETE	/api/v1/ transactions/{id}	Buchung löschen

GET Query-Parameter:

- type=INCOME | EXPENSE
- recurring=true | false (Legacy/Anzeige-Kompatibilität; vorzugsweise anhand recurringRuleId bzw. Regelstatus ableiten)
- categoryId=<uuid>
- from=YYYY-MM-DD, to=YYYY-MM-DD
- search=<Text> (Beschreibung und optional Kategoriename)
- page=0, size=20, sort=transactionDate, desc

Erstellungs-/Änderungsrequest:

```
{
  "description": "Supermarkt Einkauf",
  "amount": 92.50,
  "type": "EXPENSE",
  "categoryId": "<category-uuid>",
  "transactionDate": "2026-09-08"
```

```
}
```

Response enthält mindestens `id`, `description`, `amount`, `type`, `categoryId`, `categoryName`, `transactionDate`, `recurringRuleId`, `createdAt`, `updatedAt`.

Pagination-Response empfohlen:

```
{
  "content": [], "page": 0, "size": 20,
  "totalElements": 0, "totalPages": 0
}
```

4.3 Kategorien

Methode	Pfad	Zweck
GET	/api/v1/ categories	Kategorien des Benutzers inkl. Standardkategorien
POST	/api/v1/ categories	Kategorie anlegen
PUT	/api/v1/ categories/{id}	Name/Typ/Farbe ändern
PATCH	/api/v1/ categories/ {id}/archive	Kategorie archivieren
DELETE	/api/v1/ categories/{id}	Nur löschen, wenn keine Buchungen/Regeln referenzieren; sonst 409 oder Archivierung verlangen

Kategorie-Request:

```
{ "name": "Hobby", "type": "BOTH", "color": "#8b5cf6" }
```

Alte Transaktionen dürfen bei Umbenennung nicht ihre

Kategoriezuordnung verlieren. Keine Kategorie löschen, die noch verwendet wird, ohne definierte Reassign-/Archivierungslogik.

4.4 Wiederkehrende Einnahmen und Ausgaben

Methode	Pfad	Zweck
GET	/api/v1/ recurring-rules	Alle Regeln; Filter type, active
GET	/api/v1/ recurring-rules/ {id}	Einzelne Regel
POST	/api/v1/ recurring-rules	Regel erstellen
PUT	/api/v1/ recurring-rules/ {id}	Regel bearbeiten
PATCH	/api/v1/ recurring-rules/ {id}/activate	Aktivieren/deaktivieren
DELETE	/api/v1/ recurring-rules/ {id}	Regel entfernen; bereits erzeugte Buchungen standardmäßig behalten
POST	/api/v1/ recurring-rules/ {id}/bookings	Optional: konkrete Buchung aus Regel erzeugen, mit Duplikatschutz

Request-Beispiel:

```
{
  "description": "Monatliches Gehalt",
  "amount": 2450.00,
  "type": "INCOME",
  "categoryId": "<category-uuid>",
  "interval": "MONTHLY",
  "startDate": "2026-10-01",
}
```

```

    "endDate": null,
    "active": true
  }

```

Regeln für die Implementierung:

- Einnahmen und Ausgaben sind gleichwertige Regeltypen.
- „Fixkosten“ darf nicht auf Ausgaben beschränkt sein; UI zeigt wiederkehrende Einnahmen separat.
- Monatsäquivalent für Dashboard: $\text{MONTHLY} = \text{Betrag}$, $\text{YEARLY} = \text{Betrag} / 12$, $\text{WEEKLY} = \text{Betrag} * 52 / 12$ (oder fachlich abgestimmte Monatskonvention). Ausgabe als BigDecimal mit definierter Rundung.
- Falls automatische Buchung aktiv wird: idempotente Generierung, eindeutiger Schlüssel aus Regel-ID + Fälligkeitsdatum, keine doppelten Transaktionen nach Neustart/Retry.
- Regeländerungen sollen vergangene Transaktionen nicht rückwirkend verändern.

4.5 Dashboard / Kennzahlen

Methode	Pfad	Zweck
GET	/api/v1/dashboard/ summary	KPI-Karten und wiederkehrendes Netto
GET	/api/v1/dashboard/ cashflow	Einnahmen/Ausgaben-Verlauf für Chart.js
GET	/api/v1/dashboard/ recent-transactions? limit=5	Neueste Buchungen

Summary Query: optional `from`, `to`; für „Gesamtsaldo“ und Gesamtwerte muss die fachliche Bedeutung festgelegt werden (alle Buchungen vs. ausgewählter Zeitraum). Response-Vorschlag:

```

{
  "totalIncome": 2920.00,

```

```

    "totalExpenses": 1374.80,
    "balance": 1545.20,
    "recurringMonthlyIncome": 2450.00,
    "recurringMonthlyExpenses": 940.00,
    "recurringMonthlyNet": 1510.00,
    "currency": "EUR"
  }

```

Die Beispielwerte sind nur Schema-Beispiele, keine verbindlichen Berechnungsergebnisse.

Cashflow Response:

```

{
  "period": "MONTH",
  "points": [
    { "period": "2026-08", "income": 2450.00, "expenses": 940.00 },
    { "period": "2026-09", "income": 2920.00, "expenses": 940.00 }
  ]
}

```

4.6 Statistiken

Methode	Pfad	Zweck
GET	<code>/api/v1/statistics/overview?from=&to=</code>	Einnahmen, Ausgaben, Saldo und Fix/variabel
GET	<code>`/api/v1/statistics/by-category?type=INCOME</code>	EXPENSE&from=&to=`
GET	<code>/api/v1/statistics/recurring</code>	Aktive wiederkehrende Einnahmen/Ausgaben, Monatsäquivalente und Netto
GET	<code>`/api/v1/statistics/cashflow?from=&to=&groupBy=DAY</code>	MONTH

Beispiel `by-category`:

```
{
  "type": "EXPENSE",
  "from": "2026-09-01",
  "to": "2026-09-30",
  "items": [
    { "categoryId": "<uuid>", "categoryName": "Wohnen", "
  ]
}
```

Statistikregeln:

- Transaktionsbasierte Statistiken zählen konkrete Buchungen im gewählten Zeitraum.
- Fix/variabel basiert auf einer eindeutigen Regel: Buchung mit `recurringRuleId` gilt als wiederkehrend; ohne Verknüpfung als variabel/Einmalbuchung. Falls wiederkehrende Buchungen weiterhin nur per Flag gespeichert werden, das Flag konsistent verwenden.
- Wiederkehrende Monats-KPIs basieren auf aktiven Regeln und Monatsäquivalenten; sie sind Prognose/Planwerte und dürfen nicht stillschweigend in tatsächliche Einnahmen/Ausgaben eingerechnet werden.
- Keine doppelten Summen, wenn konkrete Buchungen bereits aus Regeln erzeugt wurden.

5. Allgemeine API-Konventionen

- 201 Created bei erfolgreichem POST, inklusive Location-Header wenn sinnvoll.
- 200 OK bei GET/PUT/PATCH, 204 No Content bei erfolgreichem DELETE.
- 400 Bad Request bei Validierungsfehlern, 401 Unauthorized bei fehlender/ungültiger Authentifizierung, 403 Forbidden bei fehlender Berechtigung, 404 Not Found bei unbekannter oder fremder Ressource (keine Existenz fremder IDs verraten), 409 Conflict bei E-Mail-/Kategorien-Duplikaten oder referenzierten Löschungen.

- `BigDecimal` in JSON als Zahl; Währungsbetrag stets EUR in der ersten Version.
- Serverseitige Validierung ist verbindlich; Frontend-Validierung dient nur UX.
- Sortierfelder serverseitig whitelisten, Pagination-Größen begrenzen.
- Benutzer-ID nie aus frei übermitteltem JSON übernehmen, sondern aus Security Principal ermitteln.

Fehlerformat (RFC-7807/Problem Details empfohlen):

```
{
  "type": "about:blank",
  "title": "Validation failed",
  "status": 400,
  "detail": "Die Anfrage enthält ungültige Felder.",
  "errors": { "amount": "Muss größer als 0 sein." }
}
```

6. Frontend-Integrationsanforderungen

1. Beim App-Start Session prüfen (`GET /auth/me`); ohne gültige Session Login zeigen.
2. Nach Login Benutzerprofil übernehmen und Dashboard-Daten laden.
3. Nach erfolgreichem Transaktions-POST/PUT/DELETE betroffene Listen, KPI-Karten, letzte Aktivitäten und Charts neu laden oder gezielt aktualisieren.
4. Kategorie-Dropdown aus `GET /categories` befüllen; nur passende/nicht archivierte Kategorien anbieten.
5. Such- und Filterzustand der Transaktionsseite als Query-Parameter ans Backend senden; Suche mit Debounce (ca. 250–400 ms) empfohlen.
6. Ladezustände, leere Ergebnisse, Netzwerkfehler und Session-Ablauf sichtbar behandeln.
7. `localStorage` nur noch für nicht-sensitive UI-Präferenzen (z. B. Theme) verwenden. Keine Transaktionsdaten oder

- Authentifizierungswahrheit dort persistieren.
8. Theme-Optionen bleiben frontendseitig; optional später `PATCH /api/v1/users/me/preferences` ergänzen, falls geräteübergreifende Synchronisierung gewünscht ist.
 9. Chart.js bleibt Frontend-Aufgabe; Backend liefert nur strukturierte Zeitreihen/Kategorien-Summen.
 10. Entfernen der Demo-Transaktionen aus dem regulären Startpfad. Demo-Daten nur über explizites Seed-/Testprofil.

7. Nicht-funktionale Anforderungen / Sicherheit

- Passwörter nur gehasht (BCrypt/Argon2), Rate-Limit/Schutz gegen Login-Bruteforce.
- HTTPS in produktiver Umgebung; Secrets aus Umgebungsvariablen/Secret Store.
- SQL-Injection-Schutz durch JPA/parametrisierte Queries; keine dynamisch ungeprüften Sortierausdrücke.
- Jeder CRUD-Endpunkt prüft Eigentümerschaft der Ressource.
- Transaktionen und wiederkehrende Regeln konsistent in DB speichern; Lösch-/Änderungsoperationen mit passenden DB-Constraints.
- Keine Finanzdaten, Passwörter oder Tokens in Logs.
- Datenbankmigrationen mit Flyway oder Liquibase.
- Unit-/Integrationstests für Berechtigungen, Betragsvalidierung, Summen, Filter, wiederkehrende Monatsäquivalente und Duplikatschutz.

8. Akzeptanzkriterien

- ☐ Benutzer kann sich anmelden und abmelden; Loginstatus wird serverseitig validiert.
- ☐ Jeder Benutzer sieht ausschließlich seine Kategorien, Transaktionen und Regeln.
- ☐ Transaktionen können erstellt, angezeigt, bearbeitet, gesucht,

gefiltert und gelöscht werden.

- ☐ Einnahmen und Ausgaben werden durch `type` unterschieden; Betrag bleibt positiv.
- ☐ Kategoriezuordnung ist validiert; verwendete Kategorien können nicht versehentlich gelöscht werden.
- ☐ Wiederkehrende Einnahmen **und** Ausgaben können angelegt, bearbeitet, deaktiviert und gelöscht werden.
- ☐ Dashboard-Summen, monatliches wiederkehrendes Netto, letzte Aktivitäten und Cashflow sind über APIs verfügbar.
- ☐ Statistiken unterstützen Zeitraumfilter, Kategorieauswertung und Fix/variabel-Aufteilung.
- ☐ Tatsächliche Buchungen und Planwerte wiederkehrender Regeln werden nicht vermischt oder doppelt gezählt.
- ☐ Fehlerantworten sind konsistent und Frontend kann Validierungsfehler feldbezogen anzeigen.
- ☐ Backend startet mit MySQL und Schema-Migrationen; Demo-Daten sind nicht Bestandteil des Produktivdatensatzes.

9. Empfohlene Implementierungsreihenfolge

1. Datenmodell, Migrationen, Authentifizierung und Benutzer-Isolation.
2. Kategorien-Endpunkte und Standardkategorien.
3. Transaktions-CRUD, Filter, Suche und Pagination.
4. Dashboard Summary/Recent/Cashflow.
5. RecurringRule CRUD für Einnahmen und Ausgaben.
6. Statistik-Endpunkte samt festgelegten Berechnungsregeln.
7. Frontend-API-Service, Fehler-/Ladezustände und Entfernung der Demo-Storage-Logik.
8. Integrationstests und OpenAPI-Dokumentation.

10. Offene fachliche Entscheidungen vor Implementierung

- Bedeutet „Gesamtsaldo“ die Summe aller jemals erfassten

Transaktionen oder einen manuell gepflegten Kontostand als Startwert plus Buchungen?

- Sollen wiederkehrende Regeln nur geplante Werte anzeigen oder automatisch konkrete Transaktionen erzeugen?
- Falls automatische Erzeugung: wann wird gebucht (Fälligkeitstag, Tagesbeginn, manuelle Bestätigung) und wie werden verpasste Termine behandelt?
- Welche Intervalle außer monatlich werden benötigt und wie wird ein Monatsäquivalent für Wochenintervalle definiert?
- Registrierung offen, nur Admin-Anlage oder zunächst ein einzelner Benutzer?
- Sollen Kategorien benutzerdefiniert sein, globale Defaults verwenden oder beides?
- Sollen Theme-Einstellungen ausschließlich lokal bleiben oder im Benutzerprofil gespeichert werden?